

STUDENT MANAGEMENT SYSTEM

C++ Modular Implementation (Fixed Array Solution)

Roll Number: 202511313

This document presents the complete, modular source code for the **Student Management System** in C++ following the specified structure requirements. In compliance with procedural guidelines, the program uses stable fixed-size standard arrays instead of standard vectors to manage system memory safely and sequentially.

1. Header File Structure: `student.h`

This file defines the core global constants, structural blueprint for student records, and interface blueprints for the core logic functions.

```
#ifndef STUDENT_H
#define STUDENT_H

#include <string>

// Maximum number of students the system can hold
const int MAX_STUDENTS = 100;

// B. Define a Student Structure
struct Student {
    int id;
    std::string name;
    int age;
    std::string course;
};

// C. Declare the Following Functions
void addStudent(Student students[], int &currentCount);
void displayStudents(const Student students[], int currentCount);
void searchStudent(const Student students[], int currentCount);
void updateStudent(Student students[], int currentCount);
void deleteStudent(Student students[], int &currentCount);

#endif // STUDENT_H
```

2. Logic Implementation: `student.cpp`

This file handles state updates, search mechanics via key structures, element shifting safely on deletion sequences, and linear validation scans.

```

#include "student.h"
#include <iostream>

using namespace std;

// D. Implement the Functions

// 1. Add a student to the array
void addStudent(Student students[], int &currentCount) {
    if (currentCount >= MAX_STUDENTS) {
        cout << "\nError: System is full. Cannot add more students.\n";
        return;
    }

    Student newStudent;
    cout << "\nEnter Student ID: ";
    cin >> newStudent.id;
    cin.ignore(); // Clear input buffer

    // Duplicate check
    for (int i = 0; i < currentCount; i++) {
        if (students[i].id == newStudent.id) {
            cout << "Error: A student with this ID already exists.\n";
            return;
        }
    }

    cout << "Enter Name: ";
    getline(cin, newStudent.name);

    cout << "Enter Age: ";
    cin >> newStudent.age;
    cin.ignore();

    cout << "Enter Course: ";
    getline(cin, newStudent.course);

    students[currentCount] = newStudent;
    currentCount++;
    cout << "Student added successfully!\n";
}

// 2. Display all registered students
void displayStudents(const Student students[], int currentCount) {
    if (currentCount == 0) {
        cout << "\nNo student records found.\n";
        return;
    }
}

```

```

    cout << "\n--- Student Records ---\n";
    for (int i = 0; i < currentCount; i++) {
        cout << "ID: " << students[i].id
            << " | Name: " << students[i].name
            << " | Age: " << students[i].age
            << " | Course: " << students[i].course << "\n";
    }
}

// 3. Search for a student by ID
void searchStudent(const Student students[], int currentCount) {
    if (currentCount == 0) {
        cout << "\nNo student records available to search.\n";
        return;
    }

    int searchId;
    cout << "\nEnter Student ID to search: ";
    cin >> searchId;

    for (int i = 0; i < currentCount; i++) {
        if (students[i].id == searchId) {
            cout << "\nStudent Found:\n";
            cout << "ID: " << students[i].id << "\n";
            cout << "Name: " << students[i].name << "\n";
            cout << "Age: " << students[i].age << "\n";
            cout << "Course: " << students[i].course << "\n";
            return;
        }
    }
    cout << "Student with ID " << searchId << " not found.\n";
}

// 4. Update student information
void updateStudent(Student students[], int currentCount) {
    if (currentCount == 0) {
        cout << "\nNo student records available to update.\n";
        return;
    }

    int updateId;
    cout << "\nEnter Student ID to update: ";
    cin >> updateId;

    for (int i = 0; i < currentCount; i++) {
        if (students[i].id == updateId) {
            cout << "\nRecord found. Enter new details:\n";
            cin.ignore();

            cout << "Enter New Name: ";

```

```

        getline(cin, students[i].name);

        cout << "Enter New Age: ";
        cin >> students[i].age;
        cin.ignore();

        cout << "Enter New Course: ";
        getline(cin, students[i].course);

        cout << "Student information updated successfully!\n";
        return;
    }
}
cout << "Student with ID " << updateId << " not found.\n";
}

// 5. Delete a student record
void deleteStudent(Student students[], int &currentCount) {
    if (currentCount == 0) {
        cout << "\nNo student records available to delete.\n";
        return;
    }

    int deleteId;
    cout << "\nEnter Student ID to delete: ";
    cin >> deleteId;

    for (int i = 0; i < currentCount; i++) {
        if (students[i].id == deleteId) {
            // Shift elements left to fill empty slot
            for (int j = i; j < currentCount - 1; j++) {
                students[j] = students[j + 1];
            }
            currentCount--; // Reduce tracking boundary counter
            cout << "Student record deleted successfully!\n";
            return;
        }
    }
    cout << "Student with ID " << deleteId << " not found.\n";
}

```

3. Application Entry Point: `main.cpp`

This file presents a continuous cyclic choice terminal interface that triggers processing branches on valid user requests.

```

#include <iostream>
#include "student.h"

using namespace std;

// E. Main Program
int main() {
    Student studentArray[MAX_STUDENTS];
    int totalStudents = 0;
    int choice;

    do {
        cout << "\n=====\\n";
        cout << "    STUDENT MANAGEMENT SYSTEM    \\n";
        cout << "=====\\n";
        cout << "1. Add Student\\n";
        cout << "2. Display All Students\\n";
        cout << "3. Search Student by ID\\n";
        cout << "4. Update Student Information\\n";
        cout << "5. Delete Student\\n";
        cout << "6. Exit\\n";
        cout << "Enter your choice (1-6): ";
        cin >> choice;

        switch (choice) {
            case 1:
                addStudent(studentArray, totalStudents);
                break;
            case 2:
                displayStudents(studentArray, totalStudents);
                break;
            case 3:
                searchStudent(studentArray, totalStudents);
                break;
            case 4:
                updateStudent(studentArray, totalStudents);
                break;
            case 5:
                deleteStudent(studentArray, totalStudents);
                break;
            case 6:
                cout << "\\nExiting program. Goodbye!\\n";
                break;
            default:
                cout << "\\nInvalid choice! Please select an option between 1 and 6.\\n";
        }
    } while (choice != 6);
}

```

```
return 0;  
}
```